

UNIVERSITÉ DU SINE SALOUM EL HADJI IBRAHIMA NIASS**UFR des Sciences Fondamentales et de l'Ingénierie****Département de Mathématiques-Informatique****Travaux Dirigés**

Algorithmique et Programmation II

*Thème : Les Listes Chaînées***Enseignant : Dr B. DIOP****Structure de base utilisée dans ce TD :**

```
typedef struct Noeud *tete; int taille; Liste;
```

Partie I : Concepts fondamentaux et comparaisons**Exercice 1. Questions de cours : Tableaux vs Pointeurs vs Listes Chaînées**

Répondre aux questions suivantes :

1. Quelle est la différence fondamentale entre un tableau et une liste chaînée en termes d'allocation mémoire ?
2. Compléter le tableau comparatif suivant :

Opération	Tableau	Pointeur	Liste chaînée
Accès à l'élément i	$O(?)$	$O(?)$	$O(?)$
Insertion en tête	$O(?)$	$O(?)$	$O(?)$
Insertion en queue	$O(?)$	$O(?)$	$O(?)$
Suppression en tête	$O(?)$	$O(?)$	$O(?)$
Recherche d'un élément	$O(?)$	$O(?)$	$O(?)$

3. Dans quels cas préférer une liste chaînée à un tableau ? Donner 3 exemples concrets.
4. Quel est l'inconvénient majeur des listes chaînées par rapport aux tableaux pour l'accès aux données ?
5. Expliquer pourquoi l'insertion en milieu de liste est plus efficace avec une liste chaînée qu'avec un tableau.

Exercice 2. Analyse de complexité

Pour chacune des opérations suivantes, donner la complexité temporelle dans le pire cas et justifier brièvement :

1. Parcourir tous les éléments d'une liste de n éléments.
2. Accéder au k -ième élément d'une liste chaînée simple.
3. Insérer un élément en tête d'une liste chaînée.

4. Insérer un élément en queue d'une liste chaînée simple (sans pointeur de queue).
5. Insérer un élément en queue d'une liste chaînée avec pointeur de queue.
6. Supprimer le dernier élément d'une liste chaînée simple.
7. Inverser une liste chaînée.

Partie II : Opérations de base sur les listes chaînées

Exercice 3. Création et initialisation

Écrire les fonctions suivantes :

```
// Crée et retourne un nouveau noeud avec la valeur donnée
Noeud* creerNoeud(int valeur);

// Initialise une liste vide
void initialiserListe(Liste *liste);

// Vérifie si la liste est vide
int estVide(Liste *liste);

// Retourne la taille de la liste
int taille(Liste *liste);
```

Exercice 4. Insertion en tête

Écrire une fonction qui insère un nouvel élément en tête de liste.

Prototype :

```
void ajouterTete(Liste *liste, int valeur);
```

Exemple :

```
Liste : 3 -> 5 -> 7 -> NULL
ajouterTete(&liste, 1);
Liste : 1 -> 3 -> 5 -> 7 -> NULL
```

Exercice 5. Insertion en queue

Écrire une fonction qui insère un nouvel élément à la fin de la liste.

Prototype :

```
void ajouterQueue(Liste *liste, int valeur);
```

Indication : Parcourir jusqu'au dernier noeud, puis ajouter.

Exercice 6. Insertion à une position donnée

Écrire une fonction qui insère un élément à la position p (0-indexé).

Prototype :

```
int ajouterPosition(Liste *liste, int valeur, int position);
```

Retour : 1 si succès, 0 si position invalide.

Exemple :

```
Liste : 1 -> 3 -> 7 -> NULL
ajouterPosition(&liste, 5, 2);
Liste : 1 -> 3 -> 5 -> 7 -> NULL
```

Exercice 7. Insertion dans une liste triée

Écrire une fonction qui insère un élément dans une liste triée en maintenant l'ordre croissant.

Prototype :

```
void ajouterTrie(Liste *liste, int valeur);
```

Exemple :

```
Liste : 1 -> 3 -> 7 -> 9 -> NULL
ajouterTrie(&liste, 5);
Liste : 1 -> 3 -> 5 -> 7 -> 9 -> NULL
```

Exercice 8. Suppression en tête

Écrire une fonction qui supprime et retourne l'élément en tête de liste.

Prototype :

```
int supprimerTete(Liste *liste);
```

Attention : Gérer le cas de la liste vide. Libérer la mémoire du noeud supprimé.

Exercice 9. Suppression en queue

Écrire une fonction qui supprime et retourne le dernier élément de la liste.

Prototype :

```
int supprimerQueue(Liste *liste);
```

Indication : Trouver l'avant-dernier noeud pour mettre son **suivant** à NULL.

Exercice 10. Suppression à une position donnée

Écrire une fonction qui supprime l'élément à la position p .

Prototype :

```
int supprimerPosition(Liste *liste, int position);
```

Exercice 11. Suppression par valeur

Écrire une fonction qui supprime la première occurrence d'une valeur dans la liste.

Prototype :

```
int supprimerValeur(Liste *liste, int valeur);
```

Retour : 1 si trouvé et supprimé, 0 sinon.

Exercice 12. Suppression de toutes les occurrences

Écrire une fonction qui supprime toutes les occurrences d'une valeur.

Prototype :

```
int supprimerToutesOccurrences(Liste *liste, int valeur);
```

Retour : Le nombre d'éléments supprimés.

Exercice 13. Affichage de la liste

Écrire deux fonctions d’affichage :

Prototypes :

```
// Affichage itératif : 1 -> 3 -> 5 -> NULL
void afficherListe(Liste *liste);

// Affichage récursif inverse : 5 -> 3 -> 1 -> NULL
void afficherInverse(Noeud *noeud);
```

Exercice 14. Libération de la mémoire

Écrire une fonction qui libère toute la mémoire allouée pour la liste.

Prototype :

```
void libererListe(Liste *liste);
```

Attention : Sauvegarder le pointeur `suivant` avant de libérer chaque noeud.

Partie III : Recherche et parcours**Exercice 15. Recherche d’un élément**

Écrire une fonction qui recherche une valeur et retourne un pointeur vers le noeud correspondant.

Prototype :

```
Noeud* rechercher(Liste *liste, int valeur);
```

Retour : Pointeur vers le noeud trouvé, ou NULL si absent.

Exercice 16. Compter les occurrences

Écrire une fonction qui compte le nombre d’occurrences d’une valeur dans la liste.

Prototype :

```
int compterOccurrences(Liste *liste, int valeur);
```

Exercice 17. Trouver le maximum et le minimum

Écrire des fonctions qui retournent les valeurs maximale et minimale de la liste.

Prototypes :

```
int maximum(Liste *liste);
int minimum(Liste *liste);
```

Exercice 18. Accès au n -ième élément

Écrire une fonction qui retourne la valeur du n -ième élément (0-indexé).

Prototype :

```
int nieme(Liste *liste, int n);
```

Question : Pourquoi cette opération est-elle moins efficace qu’avec un tableau ?

Exercice 19. Élément du milieu

Écrire une fonction qui trouve l'élément du milieu de la liste en un seul parcours.

Prototype :

```
int elementMilieu(Liste *liste);
```

Indication : Utiliser deux pointeurs : un *lent* (avance de 1) et un *rapide* (avance de 2).

Exercice 20. N -ième élément depuis la fin

Écrire une fonction qui retourne le n -ième élément depuis la fin de la liste.

Prototype :

```
int niemeDepuisFin(Liste *liste, int n);
```

Indication : Utiliser deux pointeurs avec un décalage de n positions.

Partie IV : Transformations et manipulations**Exercice 21. Inverser une liste**

Écrire une fonction qui inverse l'ordre des éléments de la liste (en place).

Prototype :

```
void inverser(Liste *liste);
```

Indication : Utiliser trois pointeurs : *precedent*, *courant*, *suivant*.

Exemple :

Avant : 1 -> 2 -> 3 -> 4 -> NULL

Après : 4 -> 3 -> 2 -> 1 -> NULL

Exercice 22. Supprimer les doublons

Écrire une fonction qui supprime les doublons d'une liste triée.

Prototype :

```
void supprimerDoublons(Liste *liste);
```

Exemple :

Avant : 1 -> 1 -> 2 -> 3 -> 3 -> 3 -> 4 -> NULL

Après : 1 -> 2 -> 3 -> 4 -> NULL

Exercice 23. Supprimer les doublons (liste non triée)

Écrire une fonction qui supprime les doublons d'une liste non triée, en gardant la première occurrence.

Prototype :

```
void supprimerDoublonsNonTrie(Liste *liste);
```

Exercice 24. Concaténer deux listes

Écrire une fonction qui concatène deux listes (la seconde à la fin de la première).

Prototype :

```
void concatener(Liste *liste1, Liste *liste2);
```

Note : Après l'opération, liste2 devient vide.

Exercice 25. Copier une liste

Écrire une fonction qui crée une copie indépendante d'une liste.

Prototype :

```
Liste* copierListe(Liste *source);
```

Exercice 26. Diviser une liste en deux

Écrire une fonction qui divise une liste en deux moitiés.

Prototype :

```
void diviserListe(Liste *source, Liste *gauche, Liste *droite);
```

Exemple :

Source : 1 -> 2 -> 3 -> 4 -> 5 -> NULL

Gauche : 1 -> 2 -> 3 -> NULL

Droite : 4 -> 5 -> NULL

Partie V : Tri et fusion

Exercice 27. Vérifier si la liste est triée

Écrire une fonction qui vérifie si la liste est triée en ordre croissant.

Prototype :

```
int estTriee(Liste *liste);
```

Exercice 28. Tri par insertion

Écrire une fonction qui trie la liste en utilisant le tri par insertion.

Prototype :

```
void triInsertion(Liste *liste);
```

Indication : Créer une nouvelle liste triée et y insérer chaque élément à sa place.

Exercice 29. Tri fusion (Merge Sort)

Écrire une fonction qui trie la liste en utilisant le tri fusion.

Prototypes :

```
void triFusion(Liste *liste);
```

```
Liste* fusionner(Liste *gauche, Liste *droite);
```

Indication : Diviser récursivement, trier, puis fusionner.

Exercice 30. Fusionner deux listes triées

Écrire une fonction qui fusionne deux listes triées en une seule liste triée.

Prototype :

```
Liste* fusionnerTriees(Liste *liste1, Liste *liste2);
```

Partie VI : Problèmes classiques**Exercice 31. Détecter un cycle**

Écrire une fonction qui détecte si la liste contient un cycle.

Prototype :

```
int contientCycle(Liste *liste);
```

Indication : Algorithme de Floyd (tortue et lièvre).

Exercice 32. Trouver le point de départ du cycle

Si la liste contient un cycle, écrire une fonction qui retourne le noeud où commence le cycle.

Prototype :

```
Noeud* debutCycle(Liste *liste);
```

Exercice 33. Vérifier si une liste est un palindrome

Écrire une fonction qui vérifie si les valeurs de la liste forment un palindrome.

Prototype :

```
int estPalindrome(Liste *liste);
```

Exemple : $1 \rightarrow 2 \rightarrow 3 \rightarrow 2 \rightarrow 1$ est un palindrome.

Exercice 34. Intersection de deux listes

Écrire une fonction qui trouve le noeud d'intersection de deux listes (si elles convergent).

Prototype :

```
Noeud* trouverIntersection(Liste *liste1, Liste *liste2);
```

Exercice 35. Additionner deux nombres représentés par des listes

Deux nombres sont représentés par des listes où chaque noeud contient un chiffre (ordre inverse).

Écrire une fonction qui retourne leur somme.

Prototype :

```
Liste* additionner(Liste *num1, Liste *num2);
```

Exemple :

num1 : 2 -> 4 -> 3 (représente 342)

num2 : 5 -> 6 -> 4 (représente 465)

Résultat : 7 -> 0 -> 8 (représente 807)

Exercice 36. Réorganiser autour d'une valeur pivot

Écrire une fonction qui réorganise la liste de sorte que tous les éléments inférieurs à x viennent avant ceux supérieurs ou égaux à x .

Prototype :

```
void partitionner(Liste *liste, int pivot);
```

Partie VII : Listes doublement chaînées

Structure :

```
typedef struct NoeudDouble {
    int valeur;
    struct NoeudDouble *precedent;
    struct NoeudDouble *suivant;
} NoeudDouble;

typedef struct {
    NoeudDouble *tete;
    NoeudDouble *queue;
    int taille;
} ListeDouble;
```

Exercice 37. Opérations de base sur liste doublement chaînée

Implémenter les fonctions suivantes pour une liste doublement chaînée :

```
void ajouterTeteDouble(ListeDouble *liste, int valeur);
void ajouterQueueDouble(ListeDouble *liste, int valeur);
int supprimerTeteDouble(ListeDouble *liste);
int supprimerQueueDouble(ListeDouble *liste);
void afficherEndroit(ListeDouble *liste);
void afficherEnvers(ListeDouble *liste);
```

Question : Quel est l'avantage d'une liste doublement chaînée pour la suppression en queue ?

Exercice 38. Inverser une liste doublement chaînée

Écrire une fonction qui inverse une liste doublement chaînée en place.

Prototype :

```
void inverserDouble(ListeDouble *liste);
```

Indication : Échanger les pointeurs `precedent` et `suivant` de chaque noeud.

Partie VIII : Listes circulaires**Exercice 39. Liste circulaire simple**

Implémenter les fonctions de base pour une liste circulaire (le dernier élément pointe vers le premier).


```
void ajouterCirculaire(ListeCirculaire *liste, int valeur);
void afficherCirculaire(ListeCirculaire *liste);
int supprimerCirculaire(ListeCirculaire *liste, int valeur);
```

Exercice 40. Problème de Josephus

N personnes sont en cercle, numérotées de 1 à N . En partant de la personne 1, on élimine chaque K -ième personne. Trouver le survivant.

Prototype :

```
int josephus(int n, int k);
```

Exemple : $n = 7, k = 3$: Élimination dans l'ordre 3, 6, 2, 7, 5, 1. Survivant : 4.

Partie IX : Applications pratiques

Exercice 41. Implémentation d'une pile avec liste chaînée

Implémenter une pile (LIFO) en utilisant une liste chaînée.

```
typedef Liste Pile;

void empiler(Pile *pile, int valeur);    // push
int depiler(Pile *pile);                 // pop
int sommet(Pile *pile);                  // top/peek
int pileVide(Pile *pile);
```

Exercice 42. Implémentation d'une file avec liste chaînée

Implémenter une file (FIFO) en utilisant une liste chaînée.

```
typedef struct {
    Noeud *tete;
    Noeud *queue;
    int taille;
} File;

void enfiler(File *file, int valeur);    // enqueue
int defiler(File *file);                 // dequeue
int premier(File *file);                 // front
int fileVide(File *file);
```

Exercice 43. Gestion d'un historique de navigation

Implémenter un système d'historique de navigation web avec les fonctions :

```
void visiterPage(Historique *h, char *url);
char* retourArriere(Historique *h);     // Bouton "Précédent"
char* retourAvant(Historique *h);       // Bouton "Suivant"
void afficherHistorique(Historique *h);
```

Indication : Utiliser une liste doublement chaînée avec un pointeur courant.

Exercice 44. Éditeur de texte simplifié

Implémenter un éditeur de texte ligne par ligne avec les opérations :

```
void insererLigne(Editeur *e, int position, char *texte);  
void supprimerLigne(Editeur *e, int position);  
void afficherDocument(Editeur *e);  
void annuler(Editeur *e);                // Ctrl+Z
```

Indication : Chaque noeud contient une ligne de texte. Utiliser une pile pour l'historique des actions.

Bon courage !